

IncidentIQ - מערכת מבוססת בינה מלאכותית לניתוח אירועי תקלות
(Incident Response) והפקת השערות לגורם התקלה

תוכן עניינים:

פרק	עמוד
מבוא	2
מטרות הפרויקט	2
תיאור המערכת	3
ארכיטקטורת המערכת	4
טכנולוגיות וכלי פיתוח	5
השימוש בבינה מלאכותית במהלך הפרויקט	5
דוגמאות לשימוש מוצלח בבינה מלאכותית	6
דוגמאות לפלטים שגויים או מטעים של הבינה המלאכותית	6
סיכונים אתיים ומקצועיים	7
אתגרים במהלך הפיתוח ופתרונם	7
שיפורים עתידיים	8
סיכום ומסקנות	8
מקורות + קישורים	9

מבוא

בשנים האחרונות מערכות תוכנה הפכו למורכבות יותר ויותר, ומורכבות זו מגדילה גם את הסיכוי לתקלות במערכות ייצור (Production). כאשר מתרחשת תקלה, צוותי פיתוח, DevOps ו-Site Reliability Engineering נדרשים לנתח במהירות כמויות גדולות של מידע ממקורות שונים, כגון לוגים, התראות ניטור, הערות פריסה (Deployment Notes) ודיווחי משתמשים. במקרים רבים המידע חלקי, סותר או אינו חד-משמעי, ולכן זיהוי מקור התקלה מהווה אתגר משמעותי.

במקביל להתפתחות מערכות התוכנה, התפתחו גם מודלי בינה מלאכותית המסוגלים לסכם מידע, לזהות דפוסים ולהציע הסברים אפשריים לבעיות מורכבות. עם זאת, שימוש לא ביקורתי בבינה מלאכותית עלול להוביל למסקנות שגויות, לביטחון מופרז בתשובות שאינן מבוססות וליצירת הנחות שאינן נתמכות בראיות. לכן, הבינה המלאכותית יכולה לשמש ככלי עזר משמעותי, אך אינה מהווה תחליף לשיקול דעת מקצועי.

במסגרת פרויקט זה פותחה מערכת בשם IncidentIQ, שמטרתה לסייע בניתוח תקלות תוכנה באמצעות שילוב יכולות של בינה מלאכותית עם תהליך חשיבה ביקורתי. המערכת מקבלת מידע ממספר מקורות הקשורים לאירוע, מנתחת אותו ומייצרת סיכום אירוע, ציר זמן, הפרדה בין עובדות להנחות, מספר השערות אפשריות לגורם התקלה, צעדי המשך מומלצים וטיטוט דוח Postmortem. בנוסף, פותחה במערכת יכולת ייחודית בשם Challenge Analysis, המבצעת ניתוח ביקורתי של תוצאות ה-AI במטרה לאתר טענות שאינן נתמכות במידע, להציע הסברים חלופיים ולהצביע על הטיות חשיבה אפשריות.

אחד העקרונות המרכזיים שהנחו את פיתוח המערכת היה יצירת כלי התומך בתהליך החקירה של מהנדסי תוכנה, ולא מחליף אותו. לכן, המערכת מציגה מספר כיווני חקירה במקביל, מפרידה בין עובדות, הנחות והשערות ומעודדת את המשתמש להמשיך באימות המידע לפני קבלת החלטה. במהלך הפיתוח הושם גם דגש על ארכיטקטורה מודולרית, המאפשרת תחזוקה נוחה, הרחבה עתידית והפרדה ברורה בין רכיבי המערכת.

מטרת הפרויקט אינה לאפשר לבינה המלאכותית לקבוע מהו גורם התקלה, אלא להמחיש כיצד ניתן להשתמש בה ככלי עזר לקבלת החלטות מושכלת תוך שמירה על חשיבה ביקורתית, אימות מידע והבחנה בין עובדות לבין השערות. גישה זו משקפת את העיקרון המרכזי של הפרויקט – שימוש בבינה מלאכותית כתמיכה בתהליך קבלת ההחלטות, ולא כמקור אמת יחיד.

מטרות הפרויקט

מטרת הפרויקט הייתה לפתח מערכת המסייעת לצוותי פיתוח ותפעול בניתוח אירועי תקלות במערכות תוכנה באמצעות שילוב של בינה מלאכותית וחשיבה ביקורתית. בניגוד למערכות המנסות לספק תשובה חד-משמעית לגורם התקלה, מטרת IncidentIQ היא לארגן את המידע הקיים, להציג מספר הסברים אפשריים ולסייע למשתמש לקבל החלטות מושכלות המבוססות על הראיות שנאספו.

במסגרת הפרויקט הוגדרו מספר יעדים מרכזיים. ראשית, פיתוח ממשק פשוט ונוח המאפשר להזין מידע ממקורות שונים, כגון תיאור האירוע, לוגים, התראות ניטור, הערות פריסה (Deployment Notes), תלונות משתמשים וקבצים משלימים בפורמט CSV, JSON או TXT. לאחר קבלת המידע, המערכת מבצעת ניתוח באמצעות מודל בינה מלאכותית ומפיקה סיכום של האירוע, ציר זמן, הפרדה בין עובדות להנחות, מספר השערות אפשריות לגורם התקלה, המלצות להמשך הבדיקה וטיטוט דוח Postmortem.

מטרה נוספת הייתה לעודד שימוש ביקורתי בבינה מלאכותית. לשם כך פותחה יכולת Challenge Analysis, המבצעת ניתוח נוסף של הפלט שנוצר ומטרתה לאתר טענות שאינן נתמכות במידע, להציע הסברים חלופיים ולהצביע על הטיות חשיבה אפשריות. יכולת זו מדגישה כי גם כאשר המודל מספק תשובות מנומקות, אין לקבל אותן כמובן מאליו ויש לבחון אותן באופן ביקורתי.

בנוסף להיבט הטכנולוגי, מטרת הפרויקט הייתה להמחיש את האופן שבו ניתן לשלב בינה מלאכותית בתהליך העבודה של מהנדסי תוכנה מבלי להחליף את שיקול הדעת האנושי. המערכת נועדה לשמש ככלי מסייע לקבלת החלטות, תוך שמירה על שקיפות, הבחנה בין עובדות להשערות והצגת רמת הוודאות של כל מסקנה.

בנוסף, אחת ממטרות הפרויקט הייתה לבנות מערכת המדמה תהליך עבודה מציאותי של צוותי פיתוח ו-DevOps. לשם כך הושם דגש על הפרדה בין שלבי איסוף המידע, עיבודו באמצעות בינה מלאכותית, אימות הממצאים והצגת תוצאות ברורות ומובנות למשתמש. בדרך זו הפרויקט אינו מתמקד רק בפיתוח יכולת טכנולוגית, אלא גם בהמחשת תהליך עבודה מקצועי ומבוסס ראיות.

תיאור המערכת

מערכת IncidentIQ פותחה כאפליקציית Web המבוססת על Streamlit, ומטרתה לספק סביבת עבודה פשוטה ונוחה לניתוח אירועי תקלות במערכות תוכנה. המערכת מאפשרת למשתמש להזין מידע ממספר מקורות שונים הקשורים לאירוע, ולאחר מכן מבצעת ניתוח באמצעות מודל בינה מלאכותית של OpenAI ומציגה את תוצאות הניתוח בצורה מסודרת וברורה.

בשלב הראשון, המשתמש מזין את המידע הרלוונטי לאירוע באמצעות מספר שדות ייעודיים: תיאור התקלה, לוגים של המערכת, הערות פריסה (Deployment Notes), התראות ממערכות ניטור ותלונות משתמשים. בנוסף, ניתן לצרף קובץ משלים בפורמט JSON, TXT או CSV, המכיל מידע נוסף העשוי לסייע בתהליך הניתוח.

לאחר הזנת הנתונים, המערכת שולחת את כלל מקורות המידע למודל הבינה המלאכותית באמצעות ה-API של OpenAI. המודל מנתח את כלל המידע כמכלול אחד ומפיק דוח מובנה הכולל סיכום של האירוע, ציר זמן, הפרדה בין עובדות להנחות, מספר השערות אפשריות לגורם התקלה, ראיות התומכות או סותרות כל השערה, המלצות להמשך החקירה וכן טיוטת דוח Postmortem. לפני הצגת התוצאות למשתמש, הפלט עובר תהליך אימות כדי לוודא שהוא עומד במבנה הנתונים שהמערכת מצפה לקבל.

אחד המאפיינים המרכזיים של המערכת הוא מנגנון Challenge Analysis. לאחר יצירת הדוח הראשוני, ניתן להפעיל ניתוח נוסף הבוחן באופן ביקורתי את תוצאות הניתוח הראשוני. מטרתו היא לזהות טענות שאינן נתמכות במידע שסופק, להציע הסברים חלופיים ולהצביע על הטיות חשיבה אפשריות העלולות להשפיע על תהליך קבלת ההחלטות. בדרך זו, המערכת אינה מסתפקת ביצירת תשובה, אלא גם מעודדת את המשתמש לבחון את איכותה ואת מידת מהימנותה.

בסיום התהליך, ניתן לייצא את כלל תוצאות הניתוח לקובץ Markdown, הכולל הן את דוח הניתוח והן את תוצאות ה-Challenge Analysis, ובכך לאפשר תיעוד מסודר של האירוע ושיתוף המידע עם חברי צוות נוספים. תהליך העבודה כולו תוכנן כך שיהיה פשוט, ברור וידידותי למשתמש, תוך שמירה על עקרונות של שקיפות, הפרדה בין עובדות להשערות ושימוש ביקורתי בבינה מלאכותית. בדרך זו המערכת מספקת כלי המסייע למהנדסי תוכנה לארגן מידע מורכב, לבחון מספר כיווני חקירה אפשריים ולתעד את תהליך הניתוח בצורה מסודרת ועקבית.

 **IncidentIQ**

AI-Powered Incident Response and Root-Cause Analysis

Incident Context

Incident Description

Describe what happened...

Application Logs

Paste application logs here...

Deployment Notes

Recent deployment details...

Monitoring Alerts

Paste monitoring/alerting data...

User Complaints

Relevant user-reported symptoms...

Upload supplemental file (.txt / .json / .csv, max 5 MB)

Upload

200MB per file • TXT, JSON, CSV

Analyze Incident

Reset

ארכיטקטורת המערכת

מערכת IncidentIQ פותחה בארכיטקטורה מודולרית, במטרה להפריד בין ממשק המשתמש, הלוגיקה העסקית, התקשורת עם שירות הבינה המלאכותית ועיבוד הנתונים. חלוקה זו מאפשרת תחזוקה נוחה יותר של הקוד, הרחבת המערכת בעתיד וביצוע בדיקות לכל רכיב בנפרד.

ממשק המשתמש פותח באמצעות Streamlit, ומאפשר למשתמש להזין את נתוני האירוע בצורה אינטואיטיבית. בנוסף, הממשק אחראי להצגת תוצאות הניתוח, הפעלת מנגנון Challenge Analysis, ייצוא הדוח לקובץ Markdown וניהול מצב העבודה של המשתמש במהלך השימוש במערכת.

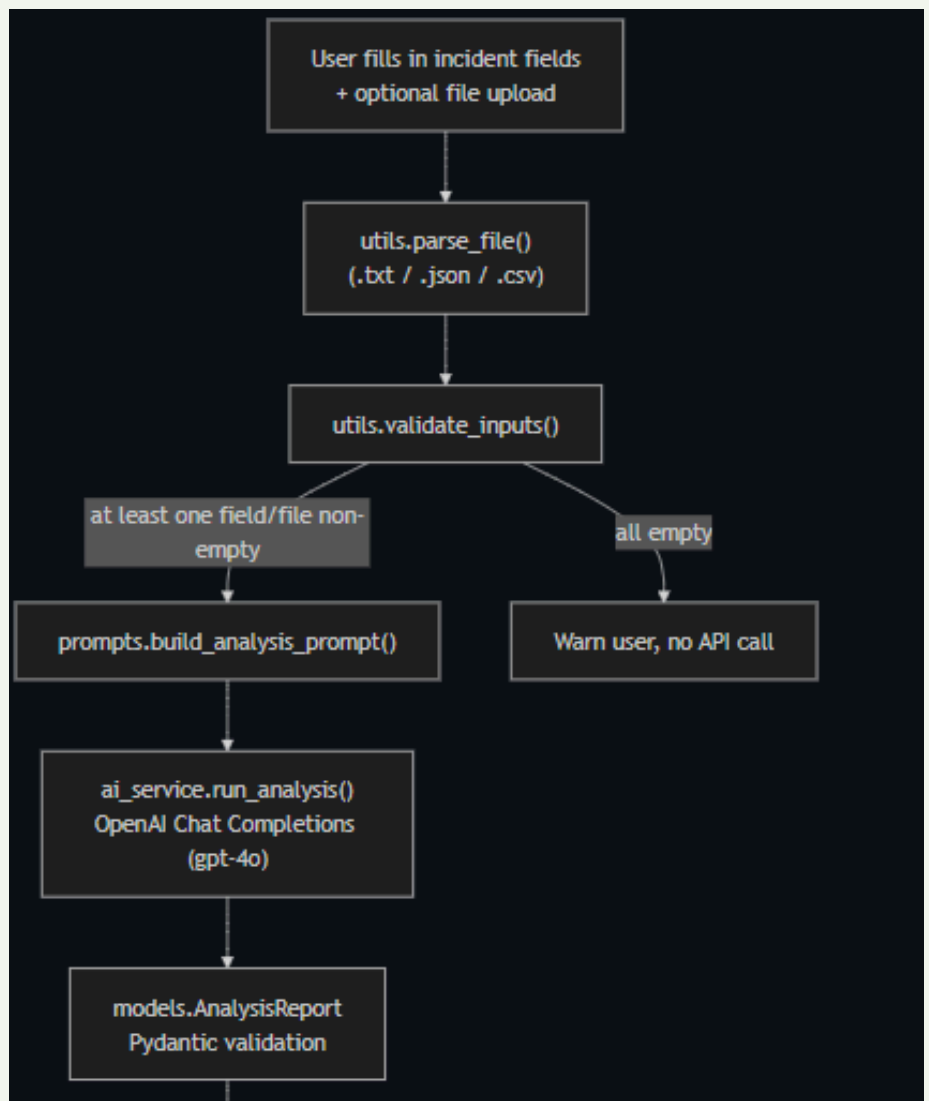
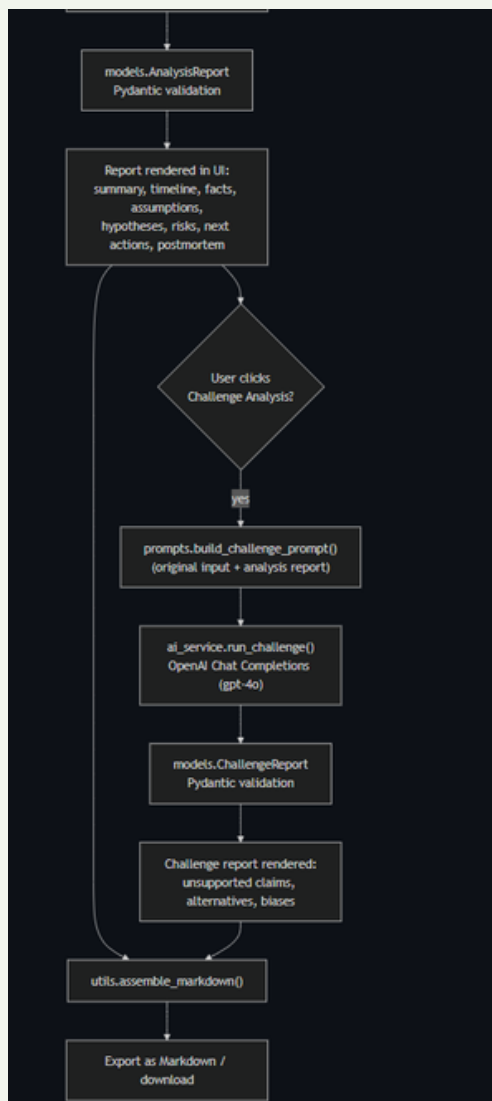
לאחר קבלת הקלט, הנתונים מועברים למודול Utils, האחראי על קריאת קבצים בפורמטים JSON, TXT ו-CSV, ביצוע עיבוד ראשוני ובדיקות תקינות של המידע, וכן יצירת דוח ה-Markdown הסופי.

בשלב הבא, מודול Prompts בונה את ההנחיות (Prompts) הנשלחות למודל הבינה המלאכותית. מודול זה אחראי על ניסוח הבקשות באופן עקבי, כך שהמודל יחזיר תשובות במבנה קבוע הכולל את כל המרכיבים הנדרשים, כגון ציר זמן, עובדות, השערות, ראיות, סיכונים וצעדי המשך.

התקשורת עם שירות הבינה המלאכותית מתבצעת באמצעות מודול AIService, אשר שולח את הבקשות ל-OpenAI API ומקבל את תשובות המודל. לאחר קבלת התשובה, היא עוברת תהליך אימות באמצעות מודלי Pydantic, המוודאים כי המידע שהוחזר תואם למבנה הנתונים שהמערכת מצפה לקבל. תהליך זה מסייע במניעת שגיאות הנובעות מפלט חלקי או מפורמט שאינו תקין, ובכך מגדיל את אמינות המערכת.

לאחר השלמת הניתוח הראשוני, המשתמש יכול להפעיל את מנגנון Challenge Analysis. בשלב זה נשלחים למודל הנתוני האירוע המקורי והן דוח הניתוח שנוצר, במטרה לבצע בחינה ביקורתית של המסקנות. תהליך זה מסייע בזיהוי טענות שאינן נתמכות במידע, הצגת הסברים חלופיים ואיתור הטיות חשיבה אפשריות.

ארכיטקטורה זו שומרת על הפרדה ברורה בין אחריות הרכיבים השונים, מפשטת את תחזוקת המערכת ומאפשרת להרחיב אותה בעתיד, למשל באמצעות הוספת מודלים נוספים, תמיכה בסוגי קבצים חדשים או שילוב יכולות נוספות, מבלי לבצע שינויים מהותיים במבנה המערכת.



טכנולוגיות וכלי הפיתוח

במהלך פיתוח הפרויקט נעשה שימוש במספר טכנולוגיות וכלי פיתוח, כאשר כל אחד מהם נבחר בהתאם לתפקידו במערכת וליתרונות שהוא מספק.

שפת התכנות המרכזית שבה פותחה המערכת היא Python, אשר אפשרה פיתוח מהיר של המערכת, שילוב נוח עם ספריות בינה מלאכותית וכתובת קוד קריא, מודולרי וקל לתחזוקה.

ממשק המשתמש פותח באמצעות Streamlit, המאפשר ליצור יישומי Web אינטראקטיביים בצורה פשוטה ומהירה. הבחירה ב-Streamlit אפשרה להתמקד בפיתוח הלוגיקה של המערכת ובחווית המשתמש, ללא צורך בפיתוח Frontend מורכב.

לצורך ביצוע הניתוחים נעשה שימוש ב-API של OpenAI, שבאמצעותו נשלחו נתוני האירוע למודל השפה לצורך הפקת הניתוח, ההשערות, ההמלצות וטיטות דוח ה-Postmortem. בנוסף, אותו שירות שימש גם להפעלת מנגנון Challenge Analysis, המבצע ניתוח ביקורתי של הפלט הראשוני.

אימות הנתונים שהוחזרו ממודל הבינה המלאכותית בוצע באמצעות Pydantic, המוודא כי מבנה הנתונים המתקבל תואם למבנה המצופה על ידי המערכת. תהליך זה מסייע במניעת שגיאות הנובעות מפלט חלקי או ממבנה נתונים שאינו תקין, ובכך תורם ליציבותה ולאמינותה של המערכת.

במהלך הפיתוח נעשה שימוש ב-Pytest לצורך כתיבה והרצה של בדיקות יחידה, וכן ב-Hypothesis לביצוע בדיקות מבוססות מאפיינים (Property-Based Testing), במטרה לוודא את תקינות הרכיבים השונים ואת עמידותם בפני מגוון רחב של קלטים.

לניהול גרסאות הקוד נעשה שימוש ב-Git, ואילו GitHub שימש לאחסון הפרויקט, תיעודו ושיתוף הקוד. בנוסף, נכתב קובץ README מפורט הכולל הוראות התקנה והרצה, תיאור המערכת, דוגמאות קלט ופלט והסבר על אופן השימוש בפרויקט, כחלק מדרישות המטלה.

בסך הכול, שילוב הטכנולוגיות שנבחרו אפשר פיתוח מערכת מודולרית, יציבה וקלה להרחבה, תוך שמירה על קוד קריא, יכולת תחזוקה גבוהה ותהליך פיתוח מסודר.

השימוש בבינה מלאכותית במהלך הפרויקט

הבינה המלאכותית הייתה מרכיב מרכזי בפיתוח המערכת ובתהליך העבודה לאורך כל הפרויקט. השימוש בה לא הסתכם ביצירת תוכן בלבד, אלא שולב במספר שלבים שונים, החל מתכנון המערכת, דרך הפיתוח בפועל ועד לבחינת איכות הפלטים שהופקו.

בשלב התכנון נעשה שימוש במודלי שפה לצורך גיבוש מבנה המערכת, חלוקת האחריות בין הרכיבים השונים ותכנון זרימת המידע. במהלך הפיתוח סייעה הבינה המלאכותית בבניית מבנה הקוד, בהצעת פתרונות לבעיות תכנות, בשיפור איכות הקוד ובהעלאת רעיונות לשיפור חוויית המשתמש.

בליבת המערכת שולב מודל השפה של OpenAI, אשר מקבל את כלל נתוני האירוע ומפיק דוח מובנה הכולל סיכום האירוע, ציר זמן, הפרדה בין עובדות להנחות, מספר השערות אפשריות לגורם התקלה, המלצות להמשך הבדיקה וטיטות דוח ה-Postmortem. כדי להבטיח שהתשובות יהיו עקביות ומסודרות, נבנו Prompts ייעודיים המגדירים למודל בצורה ברורה את מבנה הפלט המצופה ואת אופן ההצגה של כל חלק בדוח.

במהלך הפיתוח התברר כי איכות הפלט מושפעת באופן משמעותי מאופן ניסוח ההנחיות. לכן בוצעו מספר איטרציות על ה-Prompts, שבמהלכן נוספו הנחיות מפורשות להפרדה בין עובדות, הנחות והשערות, דרישה להצגת ראיות התומכות או סותרות כל השערה, וכן הנחיות לשמירה על מבנה פלט אחיד. תהליך זה שיפר את עקביות התוצאות, הפחית מקרים שבהם המודל החזיר מידע חסר או בפורמט שאינו תואם לדרישות המערכת, והקל על עיבוד התוצאות באופן אוטומטי.

לצד היתרונות של השימוש בבינה מלאכותית, היה חשוב להימנע מהסתמכות עיוורת על תשובות המודל. מסיבה זו פותח מנגנון Challenge Analysis, אשר מפעיל ניתוח נוסף על הפלט שנוצר במטרה לזהות טענות שאינן נתמכות במידע, להציע הסברים חלופיים ולהצביע על הטיות חשיבה אפשריות. מנגנון זה ממחיש את העיקרון המרכזי של הפרויקט - שימוש בבינה מלאכותית ככלי מסייע לקבלת החלטות, תוך שמירה על חשיבה ביקורתית ובדיקת מהימנות המידע.

בנוסף לשילוב מודל הבינה המלאכותית כחלק מהמערכת עצמה, נעשה שימוש בכלי בינה מלאכותית גם במהלך תהליך הפיתוח לצורך תכנון הארכיטקטורה, סקירת קוד, כתיבת תיעוד, יצירת נתוני בדיקה סינתטיים וגיבוש רעיונות לשיפור המערכת. עם זאת, בכל שלב בוצעה בחינה ידנית של ההצעות שהתקבלו, וכל החלטה תכנונית או מימושית נבדקה ואושרה על ידי צוות הפיתוח לפני ששולבה בפרויקט.

דוגמאות לשימוש מוצלח בבינה מלאכותית

במהלך פיתוח הפרויקט נעשה שימוש במודל הבינה המלאכותית במספר משימות מרכזיות, אשר תרמו באופן משמעותי לאיכות המערכת ולחווית המשתמש. עם זאת, כל תוצאות המודל נבדקו באופן ידני לפני ששולבו במערכת. אחת היכולות הבולטות של המערכת היא יצירת סיכום אוטומטי של האירוע. במקום שהמשתמש יידרש לקרוא עשרות שורות של לוגים, התראות ודיווחי משתמשים, המודל מפיק תיאור קצר וברור של האירוע, תוך זיהוי המרכיבים המרכזיים והצגתם בשפה מקצועית. יכולת זו מאפשרת להבין במהירות את תמונת המצב הראשונית. יכולת נוספת שבה הבינה המלאכותית הוכיחה את עצמה היא בניית ציר זמן (Timeline). המודל הצליח לזהות אירועים משמעותיים מתוך מקורות מידע שונים ולסדר אותם בסדר כרונולוגי. לדוגמה, המערכת קישרה בין מועד ביצוע הפריסה, הופעת ההתראות הראשונות ועלייה במספר תלונות המשתמשים, ובכך סיפקה תמונה ברורה של השתלשלות האירועים.

בנוסף, המערכת מפיקה מספר השערות אפשריות לגורם התקלה, כאשר לכל השערה מוצגים גורמים התומכים בה, גורמים הסותרים אותה, רמת ביטחון והמלצות להמשך הבדיקה. גישה זו מעודדת בחינה של מספר כיווני חקירה במקביל, במקום להסתמך על הסבר יחיד כבר בתחילת התהליך.

גם טיוטת ה-Postmortem שנוצרה על ידי המודל סיפקה בסיס טוב לדוח מסכם של האירוע. במקום להתחיל את הכתיבה מאפס, התקבלה טיוטה מסודרת שכללה את עיקרי האירוע, השפעת התקלה על המערכת, צעדי הפתרון והלקחים שניתן להפיק. לאחר בדיקה ועדכון במידת הצורך, ניתן להשתמש בטייטה זו כחלק מתהליך התיעוד ולחסוך זמן עבודה משמעותי.

באופן כללי, השימוש בבינה המלאכותית אפשר לקצר את זמן הניתוח, לארגן מידע רב בצורה ברורה ולהציע כיווני חקירה שלא תמיד היו עולים באופן מיידי. עם זאת, לאורך כל הפרויקט נשמר העיקרון שלפיו תוצאות המודל מהוות נקודת פתיחה לניתוח ולא תחליף לשיקול דעת מקצועי של המשתמש.

דוגמאות לפלטים שגויים או מטעים של הבינה המלאכותית

לצד היתרונות הרבים של השימוש בבינה מלאכותית, במהלך הפרויקט התברר כי המודל אינו חסין מטעויות. במקרים מסוימים הוא הציג מסקנות שנשמעו אמינות ומשכנעות, אך לא היו מבוססות באופן מלא על המידע שסופק לו. תופעה זו חיזקה את ההבנה כי אין להסתמך על תשובות המודל ללא בדיקה ביקורתית.

אחת הדוגמאות הבולטות הופיעה במהלך יצירת טיוטת ה-Postmortem. המודל ציין כי "השגיאות החלו זמן קצר לאחר הפריסה בשעה 14:00", בעוד שבפועל, על פי הנתונים שסופקו למערכת, השגיאה הראשונה תועדה רק בשעה 14:09. אמנם הפריסה בוצעה בשעה 14:00, אך לא ניתן להסיק באופן חד-משמעי שהתקלה החלה מיד לאחריה. מנגנון Analysis Challenge זיהה כי טענה זו אינה נתמכת באופן מלא במידע והצביע על כך שהמודל הסיק מסקנה רחבה יותר מהראיות שעמדו לרשותו.

דוגמה נוספת הייתה הנטייה של המודל להתמקד במהירות בשינוי שבוצע בגודל מאגר החיבורים למסד הנתונים (Database Connection Pool) ולהציג אותו כהסבר הסביר ביותר לתקלה. אף שהשערה זו נתמכה במספר ראיות, היא לא הייתה ההסבר האפשרי היחיד. מנגנון Analysis Challenge הציע גם אפשרות של בעיית רשת או שינוי תצורה חיצוני, אשר עשויים להסביר חלק מהתסמינים שנצפו. בכך הודגש כי גם כאשר השערה מסוימת נראית סבירה, חשוב להמשיך לבחון הסברים נוספים לפני קבלת מסקנה.

במהלך בחינת הפלטים ניתן היה לזהות מספר הטיות חשיבה אופייניות. המודל הפגין Confirmation Bias, כאשר נטה להעניק משקל גבוה יותר לראיות שתמכו בהשערה הראשונית ופחות לראיות שסתרו אותה. בנוסף, זוהתה Recency Bias, שבאה לידי ביטוי בכך שהמודל ייחס חשיבות רבה במיוחד לשינויים האחרונים שבוצעו במערכת. במקרים מסוימים ניתן היה להבחין גם ב-Automation Bias, כאשר הניסוח המקצועי של תשובות המודל יצר תחושת ביטחון גבוהה מדי במסקנותיו, וכן ב-Anchoring Bias, שהתבטאה בהתמקדות באירוע הראשון שזוהה במהלך הניתוח. בחלק מהמקרים הופיעה גם Overconfidence Bias, כאשר המודל הציג השערות ברמת ביטחון גבוהה למרות שהראיות לא אפשרו להסיק מסקנה חד-משמעית. שילוב מנגנון Analysis Challenge, יחד עם הפרדה בין עובדות, הנחות והשערות, סייע לזהות הטיות אלו ולעודד בחינה ביקורתית של הפלטים.

מקרים אלו מדגישים כי למרות יכולתה של הבינה המלאכותית לנתח מידע במהירות ולהציע כיווני חקירה, היא אינה מהווה מקור אמת מוחלט. תפקידה הוא לסייע למהנדס לארגן מידע ולהעלות השערות אפשריות, בעוד שהאחריות על אימות הממצאים וקבלת ההחלטות נותרת בידי האדם. אחת המטרות המרכזיות של הפרויקט הייתה להמחיש עיקרון זה, ולכן שולב במערכת מנגנון Analysis Challenge, המשמש כשכבת בקרה נוספת על איכות הפלטים שמפיק המודל.

סיכונים אתיים ומקצועיים

השימוש בבינה מלאכותית לצורך ניתוח אירועי תקלות עשוי לספק ערך משמעותי למהנדסי תוכנה, אך הוא מעלה גם מספר סיכונים מקצועיים ואתיים שיש להביא בחשבון. אחד הסיכונים המרכזיים הוא הסתמכות מופרזת על תשובות המודל. מכיוון שהתשובות מנוסחות בצורה מקצועית ובטוחה, קיימת נטייה לקבל אותן כנכונות גם כאשר הן אינן נתמכות במידע שהוזן למערכת. מסיבה זו, חשוב לראות בבינה המלאכותית כלי מסייע בלבד ולא מקור סמכות בקבלת החלטות הנדסיות.

סיכון נוסף הוא שליחת מידע רגיש לשירותי בינה מלאכותית חיצוניים. לוגים של מערכות ייצור עשויים להכיל מידע עסקי, נתוני משתמשים, מזהים פנימיים או מידע אבטחתי. לכן, במערכות אמיתיות יש לבצע סינון או אנונימיזציה של המידע לפני שליחתו למודל חיצוני, וכן לוודא עמידה במדיניות הארגון ובדרישות אבטחת המידע. בנוסף, יש לוודא כי השימוש בשירותי בינה מלאכותית עומד בדרישות הרגולציה ובנהלי הארגון בכל הנוגע לשמירה על פרטיות המידע. חשוב להבהיר כי האחריות לקבלת ההחלטות נשארת בידי מהנדס התוכנה ולא בידי המערכת. IncidentIQ אינה קובעת מהו גורם התקלה, אלא מציגה מספר השערות אפשריות, ראיות התומכות והסותרות כל אחת מהן והמלצות להמשך הבדיקה. בדרך זו נשמר תהליך קבלת החלטות המבוסס על שיקול דעת אנושי ולא על הסתמכות עיוורת על הבינה המלאכותית.

מעבר לכך, פיתוח מערכות המבוססות על בינה מלאכותית מחייב שקיפות לגבי מגבלות המודל והימנעות מהצגת השערות כעובדות. מסיבה זו תוכננה המערכת כך שתפריד באופן ברור בין עובדות, הנחות והשערות, ותשלב את מנגנון Analysis Challenge, המעודד בחינה ביקורתית של תוצאות הניתוח ומסייע למשתמש לזהות מסקנות שאינן נתמכות באופן מלא בראיות.

אתגרים במהלך הפיתוח ופתרונם

במהלך פיתוח הפרויקט התמודדתי עם מספר אתגרים טכנולוגיים ותכנוניים, אשר דרשו ממני לבצע התאמות ושיפורים לאורך הדרך. חלק מהאתגרים נבעו מהעבודה עם מודלי בינה מלאכותית, בעוד שאחרים היו קשורים לתכנון המערכת, לארגון הקוד ולשמירה על חוויית משתמש נוחה וברורה.

אחד האתגרים המרכזיים היה יצירת פלט עקבי ממודל השפה. בתחילת הפיתוח התקבלו לעיתים תשובות שהיו שונות במבנה שלהן, חסרו חלקים מסוימים או לא תאמו את הפורמט שהמערכת ציפתה לו. כדי להתמודד עם בעיה זו שיפרתי בהדרגה את ה־Prompts, הוספתי הנחיות מדויקות לבין אימות הנתונים שיפר משמעותית את עקביות הפלטים. הצגתם למשתמש. שילוב בין הנחיות מדויקות לבין אימות הנתונים שיפר משמעותית את עקביות הפלטים.

אתגר נוסף היה מציאת האיזון בין כמות המידע המוצגת לבין קריאות הדוח. בתחילה התקבלו תשובות ארוכות ומפורטות מדי, אשר הקשו על המשתמש לזהות במהירות את עיקרי המידע. בעקבות זאת שיפרתי את מבנה הדוחות, חילקתי את המידע לפרקים ברורים והדגשתי את המרכיבים החשובים ביותר, כגון סיכום האירוע, ציר הזמן וההמלצות.

אתגר משמעותי נוסף היה ההתמודדות עם העובדה שמודל הבינה המלאכותית עשוי להציג מסקנות שנשמעות משכנעות, אך אינן נתמכות באופן מלא במידע שסופק לו. מתוך אתגר זה פיתחתי את מנגנון Analysis Challenge, אשר מבצע ניתוח ביקורתי של הפלט, מסייע בזיהוי טענות שאינן מבוססות, מציע הסברים חלופיים ומצביע על הטיות חשיבה אפשריות. מנגנון זה הפך לאחד המרכיבים המרכזיים והייחודיים של המערכת, והוא מייצג את העיקרון המרכזי של הפרויקט - שימוש בבינה מלאכותית ככלי מסייע, ולא כמקור אמת יחיד.

במקביל, הקפדתי על פיתוח ארכיטקטורה מודולרית, כך שלכל רכיב במערכת תהיה אחריות מוגדרת. גישה זו הקלה על תהליך הפיתוח, אפשרה כתיבת בדיקות בצורה מסודרת והפכה את המערכת לפשוטה יותר לתחזוקה ולהרחבה בעתיד.

מעבר לאתגרים הטכנולוגיים, אחד האתגרים המשמעותיים ביותר היה ניהול הזמן. **במסגרת המטלה הוקצו 13 ימים בלבד לתכנון**, פיתוח, בדיקות, כתיבת התיעוד והגשת הפרויקט. מגבלת זמן זו חייבה אותי לעבוד בצורה מסודרת וממוקדת, תוך קביעת סדרי עדיפויות ברורים. כבר בתחילת העבודה חילקתי את הפרויקט למשימות יומיות, ובכל יום הגדרתי לעצמי מטרות ברורות, כגון תכנון הארכיטקטורה, פיתוח רכיבים חדשים, כתיבת בדיקות, שיפור ממשק המשתמש או השלמת חלקים מהתיעוד. עבודה מסודרת לפי יעדים יומיים אפשרה לי לעקוב אחר ההתקדמות, לזהות עיכובים בזמן ולוודא שכל שלבי הפרויקט מקבלים את הזמן הראוי להם. מעבר לכך, תהליך זה חיזק אצלי את ההבנה כי ניהול זמן נכון הוא חלק בלתי נפרד מפיתוח תוכנה, במיוחד בפרויקטים בעלי לוחות זמנים קצרים.

בסופו של דבר, האתגרים שניצבו בפניי תרמו לשיפור המערכת ולגיבוש פתרונות יציבים יותר. תהליך ההתמודדות עמם חיזק את ההבנה כי פיתוח מערכת המבוססת על בינה מלאכותית אינו מסתכם בשילוב מודל שפה בלבד, אלא מחייב תכנון נכון, ניהול זמן יעיל, מנגנוני בקרה, בדיקות וחשיבה ביקורתית לאורך כל שלבי הפיתוח.

שיפורים עתידיים

למרות שהמערכת מממשת את כלל הדרישות המרכזיות של הפרויקט, קיימים מספר כיווני פיתוח שיכולים להרחיב את יכולותיה בעתיד ולהפוך אותה לכלי מתקדם ושימושי יותר עבור צוותי פיתוח. אחד השיפורים האפשריים הוא תמיכה בניתוח מספר אירועי תקלה במקביל, כך שניתן יהיה להשוות בין אירועים דומים, לזהות דפוסים חוזרים ולסייע בזיהוי תקלות בעלות מאפיינים משותפים. בנוסף, ניתן לשלב יכולת שיחה (Chat) עם נתוני האירוע, שתאפשר למשתמש לשאול שאלות המשך לגבי הניתוח ולקבל הסברים ממוקדים על הממצאים שהתקבלו.

כיוון פיתוח נוסף הוא שילוב מנגנון Retrieval, אשר יאתר באופן חכם רק את המידע הרלוונטי מתוך לוגים גדולים במיוחד, במקום לשלוח את כלל הנתונים למודל הבינה המלאכותית. גישה זו עשויה לשפר את איכות הניתוח, לקצר את זמן העיבוד ולהפחית את כמות המידע המיותר הנשלח למודל. בנוסף, ניתן להרחיב את המערכת כך שתתמוך במספר מודלי בינה מלאכותית ותאפשר להשוות בין הפלטים שלהם. השוואה כזו עשויה לסייע בזיהוי חוסר עקביות בין המודלים, להפחית הטיות ולשפר את מהימנות התוצאות. לבסוף, ניתן לשלב את המערכת עם כלי DevOps נפוצים, כגון Jira, GitHub ומערכות ניטור, כך שנתוני האירוע יאספו באופן אוטומטי ממספר מקורות ללא צורך בהזנה ידנית. שילוב כזה יהפוך את תהליך הניתוח למהיר, יעיל ורציף יותר, ויאפשר להשתמש במערכת כחלק מתהליך העבודה היומיומי של צוותי פיתוח ותפעול.

סיכום ומסקנות

בפרויקט זה פיתחתי את IncidentIQ, מערכת המסייעת בניתוח אירועי תקלות באמצעות מודל בינה מלאכותית. מטרת המערכת הייתה לקצר את זמן הניתוח הראשוני של אירועים, לארגן מידע ממספר מקורות בצורה ברורה ולהציע למשתמש השערות והמלצות להמשך החקירה, תוך שמירה על גישה ביקורתית כלפי תוצאות המודל. במהלך הפיתוח למדתי כי שילוב בינה מלאכותית במערכות תוכנה יכול להעניק ערך משמעותי למשתמש, במיוחד במשימות הכוללות עיבוד מידע רב בזמן קצר. יחד עם זאת, הבנתי כי אין לראות בתשובות המודל אמת מוחלטת. גם כאשר הפלט נראה משכנע ומנוסח היטב, הוא עלול לכלול הנחות שאינן מבוססות או להתעלם מהסברים חלופיים. אחת התרומות המרכזיות של הפרויקט היא שילוב מנגנון Analysis Challenge, אשר בוחן באופן ביקורתי את הפלט שנוצר על ידי המודל. מנגנון זה מעודד את המשתמש לבחון את המסקנות לעומק, לזהות טענות שאינן נתמכות בראיות ולהימנע מהסתמכות עיוורת על תוצאות הבינה המלאכותית. לדעתי, שילוב של יכולות ניתוח מתקדמות לצד מנגנון ביקורת מהווה גישה מאוזנת ואחראית יותר לשימוש במודלי שפה.

מעבר לתוצר הסופי, הפרויקט תרם גם להתפתחותי כמפתח. במהלך העבודה התנסיתי בתכנון מערכת מודולרית, בעבודה עם ממשקי API, בפיתוח ממשק משתמש, בכתיבת בדיקות, בניהול גרסאות באמצעות Git ובשילוב כלי בינה מלאכותית כחלק מתהליך הפיתוח. התנסות זו חיזקה את ההבנה שלי לגבי היתרונות, המגבלות והאחריות הכרוכה בפיתוח מערכות מבוססות AI.

מעבר להיבט הטכנולוגי, נהיתי מאוד לקחת חלק בפרויקט מאתגר מסוג זה. לאורך תהליך הפיתוח הרגשתי שאני מתמודד עם משימות הדומות לאלו שמפתחים פוגשים בעולם התעשייה – החל מתכנון הארכיטקטורה, דרך עבודה עם שירותי API, תכנון ממשק משתמש, כתיבת בדיקות, פתרון בעיות ושיפור מתמיד של המערכת, ועד לניהול זמן ועמידה בלוחות זמנים. החוויה כולה העניקה לי ניסיון מעשי משמעותי והמחישה לי כיצד נראה תהליך פיתוח של מערכת אמיתית מתחילתו ועד סופו.

לסיכום, אני מאמין כי IncidentIQ ממחישה כיצד ניתן לשלב בינה מלאכותית בצורה יעילה ואחראית בתהליך ניתוח אירועים. המערכת אינה מחליפה את שיקול הדעת של איש המקצוע, אלא מספקת לו כלי עזר המסייע בארגון המידע, בהעלאת כיווני חקירה ובהפקת תובנות ראשוניות. מבחינתי, אחד הלקחים החשובים ביותר מהפרויקט הוא שהאתגר האמיתי אינו לגרום לבינה המלאכותית לענות על שאלות, אלא לדעת כיצד להשתמש בה בצורה ביקורתית, אחראית ומושכלת. אני מסיים את הפרויקט בתחושת סיפוק גדולה, מתוך אמונה שהידע והניסיון שצברתי במהלכו יהוו עבורי בסיס משמעותי להמשך דרכי המקצועית בעולם פיתוח התוכנה והבינה המלאכותית.

References

1. OpenAI. (2025). OpenAI API Documentation. Retrieved from <https://platform.openai.com/docs>
2. OpenAI. (2025). API Reference. Retrieved from <https://platform.openai.com/docs/api-reference>
3. Streamlit. (2025). Streamlit Documentation. Retrieved from <https://docs.streamlit.io/>
4. Pydantic. (2025). Pydantic Documentation. Retrieved from <https://docs.pydantic.dev/>
5. Pytest Development Team. (2025). Pytest Documentation. Retrieved from <https://docs.pytest.org/>
6. Hypothesis Works. (2025). Hypothesis Documentation. Retrieved from <https://hypothesis.readthedocs.io/>
7. GitHub. (2025). GitHub Documentation. Retrieved from <https://docs.github.com/>
8. OpenAI. (2025). ChatGPT. Retrieved from <https://chatgpt.com/>
9. Anthropic. (2025). Claude. Retrieved from <https://claude.ai/>

GitHub Repository:

- <https://github.com/yairkr13/IncidentIQ.git>

קישור לסרטון:

- https://drive.google.com/drive/folders/14nMEaURD89k2PsXiOFuscYWM1e5XExcd?usp=share_link

מגיש:

• יאיר קרוטהמר

ת"ז:

• 319010195